🔒 CONFIDENCIAL

Reporte de Vulnerabilidad

XSS almacenado con ejecución de acciones en la sesión de la víctima (publicación forzada de comentarios)

ID del Ticket	**VULN-2026-013**
Activo evaluado	**Aplicación web «El Blog de Pepe» (Lab Blog)**
Severidad	**ALTA** **CVSS v3.1 8.8**
Tipo	**Stored XSS → acción forzada en sesión ajena — CWE-79 / CWE-352**
Cliente	**Software Seguro — Laboratorio El Blog de Pepe**
Analista	**J. M. F. · jmf01111974@gmail.com**
Fecha del reporte	**09 de noviembre de 2026**
Versión	**1.0**

Contenido

1. Resumen Ejecutivo pág. 2
2. Detalles Técnicos pág. 2
3. Métricas CVSS pág. 4
4. Prueba de Concepto (PoC) pág. 5
5. Recomendaciones de Remediación pág. 6
6. Referencias e Historial pág. 8

Este documento contiene información sensible sobre vulnerabilidades de seguridad. Su distribución está restringida al personal autorizado del cliente. Las pruebas se realizaron sobre un entorno de laboratorio con autorización explícita.

1. Resumen Ejecutivo

> Audiencia: dirección y responsables de negocio. Redactado sin tecnicismos.

¿Qué encontramos?

El blog **guarda los comentarios y los vuelve a mostrar sin neutralizarlos**. Eso permite dejar un comentario que, en vez de texto, contiene código que se ejecuta en el navegador de **cualquiera que lo vea**. Se demostró dejando un comentario malicioso que, al ser visto por el dueño del blog, **publicó automáticamente un comentario en su nombre** con el texto que buscábamos.

¿Por qué importa para el negocio?

Es un **XSS almacenado** (persistente), la variante más peligrosa: queda guardado y afecta a todos los visitantes sin que el atacante haga nada más. Como el código se ejecuta en la sesión de la víctima, puede **realizar acciones en su nombre** (aquí, publicar un comentario como el dueño), **robar su sesión**, suplantarla o alterar el contenido. En un entorno real equivale a la **toma de control de cuentas** —incluidas las de administradores— y a la publicación de contenido falso atribuido a terceros, con el consiguiente daño de reputación, fraude y responsabilidad legal.

¿Qué tan grave es?

Severidad ALTA — CVSS 8.8. La explotación es sencilla: basta con dejar un comentario preparado. Cuando la víctima (el usuario `pepe`) visualizó el blog, el código publicó por él el comentario objetivo ("Voy a derogar la ley 1542-A"), obteniéndose el código de verificación del laboratorio.

¿Qué hay que hacer?

Aplicar **codificación de salida** sobre el contenido de los comentarios (que el texto se muestre como texto), reforzar con una política de seguridad de contenido (CSP), **tokens anti-CSRF** en las acciones que cambian estado y cookies protegidas.

Riesgo	Probabilidad	Impacto	Urgencia de acción
Ejecución de acciones y toma de control de cuentas (incl. privilegiadas) por XSS almacenado	Alta	Alto	**Inmediata**

2. Detalles Técnicos

> Audiencia: equipo de desarrollo y administradores de la plataforma.

2.1 Descripción de la vulnerabilidad

El módulo de comentarios almacena el texto enviado por el usuario y lo renderiza posteriormente en la página **sin codificación de salida** ni saneamiento. Esto constituye un **Cross-Site Scripting (XSS) almacenado/ persistente** (CWE-79): el payload queda guardado y se ejecuta en el navegador de cada usuario que visualiza el comentario.

El impacto se materializa porque el script inyectado realiza una **petición autenticada en nombre de la víctima**: usando `fetch()` con `credentials:'include'`, envía un `POST` a `/comentarios.php` que publica un comentario como el

usuario que está viendo la página. Al ejecutarse en la sesión del dueño del blog (`pepe`), publica contenido a su nombre. Es un XSS encadenado con una **acción de cambio de estado forzada** (patrón CSRF ejecutado desde el propio sitio), lo que además evade protecciones CSRF basadas en origen.

2.2 Flujo del ataque

- El atacante, con una cuenta con permiso de comentar (`teny`), publica un comentario que contiene el payload XSS.
- El payload queda **almacenado** y se ejecuta automáticamente cuando alguien visualiza el blog.
- Cuando la víctima privilegiada (`pepe`) abre la página, el script dispara un `POST` autenticado a `/comentarios.php` con su sesión.
- Se publica un comentario como `pepe` ("Voy a derogar la ley 1542-A"); se cumple el objetivo y se obtiene el código.

2.3 Clasificación

Categoría	Referencia
Tipo (CWE)	CWE-79 — Cross-site Scripting (Stored) · CWE-116 — Improper Encoding or Escaping of Output · CWE-352 — Cross-Site Request Forgery (acción de estado forzada)
Subtipo	Stored/Persistent XSS con ejecución de acciones autenticadas en la sesión de la víctima
OWASP Top 10 (2021)	A03:2021 — Injection (incluye Cross-Site Scripting)
OWASP WSTG	WSTG-INPV-02 — Testing for Stored Cross Site Scripting
CVE	No aplica (defecto propio de la aplicación, no de un componente de terceros)

2.4 Activos y alcance afectado

Elemento	Detalle
Aplicación	«El Blog de Pepe» (Lab Blog)
Host	`chl-<instancia>-blog-pepe.softwareseguro.com.ar` (el subdominio se asigna por instancia/sesión)
Punto vulnerable	Módulo de comentarios (campo `txtComentario`), almacenado y renderizado sin escapar
Endpoint de acción	`POST /comentarios.php` (sin protección anti-CSRF efectiva)
Impacto sobre	Sesión y cuenta de cualquier usuario que vea el comentario, incluido el dueño del blog
Alcance	Ejecución en el contexto del sitio: acciones, sesión y datos de la víctima

2.5 Causa raíz

La aplicación **inserta el contenido de los comentarios en el HTML sin codificación de salida**, permitiendo que el navegador ejecute marcado activo. A ello se suma que la acción de publicar comentarios (`/comentarios.php`) **no exige un token anti-CSRF** y la sesión viaja en una cookie utilizable por script, por lo que un XSS puede

ejecutar acciones autenticadas en nombre de la víctima. Faltan, además, controles de defensa en profundidad (CSP, cookies `HttpOnly`).

3. Métricas CVSS

Puntuación objetiva y reproducible del riesgo técnico, según CVSS v3.1.

CVSS Base Score	Severidad	Vector
8.8	ALTA	`CVSS:3.1/AV:N/AC:L/PR:L/UI:R/S:C/C:H/I:H/A:N`

Desglose del vector (métricas base)

Métrica	Valor	Justificación
Attack Vector (AV)	**Network (N)**	Se explota de forma remota a través de Internet (HTTP/HTTPS).
Attack Complexity (AC)	**Low (L)**	No requiere condiciones especiales: publicar un comentario con el payload.
Privileges Required (PR)	**Low (L)**	Se requiere una cuenta con permiso para comentar (p. ej. `teny`).
User Interaction (UI)	**Required (R)**	La víctima debe visualizar el comentario almacenado para que el payload se ejecute.
Scope (S)	**Changed (C)**	El código se ejecuta en el navegador/sesión de la víctima, un ámbito distinto al del servidor.
Confidentiality (C)	**High (H)**	Posible robo de sesión/cookies y datos de la víctima en el contexto del sitio.
Integrity (I)	**High (H)**	Ejecuta acciones autenticadas en nombre de la víctima (publica como el dueño del blog).
Availability (A)	**None (N)**	No afecta la disponibilidad del servicio.

Nota de criterio: la valoración Alta refleja un XSS **almacenado** capaz de ejecutar acciones autenticadas y comprometer cuentas (incl. privilegiadas). Si el impacto se limitara a robo de sesión/cookies (`HttpOnly` ausente) sin acciones, se mantendría en rango Alto/Medio; la capacidad demostrada de actuar como la víctima sostiene `C:H/I:H`.

Verificable en la calculadora oficial: https://www.first.org/cvss/calculator/3.1

4. Prueba de Concepto (PoC)

Reproducción paso a paso. Cualquier técnico con una cuenta del laboratorio puede replicar el hallazgo.

4.1 Requisitos previos

- Cuenta con permiso para comentar: usuario `teny` / clave `Teny1805`.
- Un navegador web o Burp Suite.
- El mecanismo del laboratorio para que la víctima (`pepe`) visite el blog (ingeniería social provista; no es parte de la vulnerabilidad evaluada).

4.2 Pasos de reproducción

Paso 1 — Autenticarse y localizar el campo de comentarios. Iniciar sesión como `teny` y confirmar que el contenido de un comentario se refleja/renderiza al mostrarse.

Paso 2 — Publicar el comentario con el payload XSS almacenado. En el campo de comentario (`txtComentario`) insertar un `<img>` cuyo manejador `onerror` ejecuta un `fetch()` autenticado que publica el comentario objetivo.

```
<img src=x onerror="fetch('/comentarios.php',{method:'POST',credentials:'include',
  headers:{'Content-Type':'application/x-www-form-urlencoded'},
  body:'txtComentario=Voy%20a%20derogar%20la%20ley%201542-A&btnEnviar=Enviar'})">
```

Evidencia 1 — El comentario se guarda y, al renderizarse, el `onerror` ejecuta el `fetch()`: XSS almacenado confirmado.

Paso 3 — Provocar la visita de la víctima. Mediante la funcionalidad del laboratorio, hacer que `pepe` ingrese al blog (se le pasa el dominio del desafío).

Paso 4 — Ejecución en la sesión de la víctima. Al cargar la página, el navegador de `pepe` ejecuta el payload y envía el `POST` a `/comentarios.php` con **su** sesión (`credentials:'include'`), publicando el comentario a su nombre.

Evidencia 2 — Aparece un comentario de `pepe` con el texto "Voy a derogar la ley 1542-A", publicado sin su intervención consciente.

Paso 5 — Verificación del impacto. Actualizar la página: el comentario forzado está publicado como la víctima y el laboratorio entrega el código.

```
# Código de verificación obtenido:
#   c4c309a13c8fc4c5f48e72e4154dc812
```

Evidencia 3 — Objetivo cumplido: se ejecutó una acción autenticada en nombre de la víctima mediante XSS almacenado.

> Nota técnica: el mismo XSS permite acciones de mayor impacto (robo de `document.cookie` si la sesión no es `HttpOnly`, exfiltración de datos, cambios de configuración de la cuenta). La PoC se limitó a publicar el comentario objetivo.

4.3 Resultado esperado vs. obtenido

Esperado (sistema seguro)	Obtenido (sistema vulnerable)
El comentario se almacena y muestra como texto (HTML escapado); un `<img onerror>` no se ejecuta, y `/comentarios.php` exige token anti-CSRF.	El comentario ejecuta JavaScript al visualizarse y publica una acción autenticada en nombre de la víctima.

> ⚠ Nota ética: la PoC se ejecutó únicamente sobre el entorno de laboratorio autorizado. No se atacó la funcionalidad de ingeniería social (provista por el laboratorio) ni se realizaron acciones fuera del objetivo del desafío.

5. Recomendaciones de Remediación

> Audiencia: equipo de desarrollo que implementará la corrección.

5.1 Solución recomendada (prioritaria)

Codificación de salida de los comentarios. El contenido almacenado debe escaparse al renderizarse, de modo que el marcado se muestre como texto y no se ejecute. Usar el autoescapado del motor de plantillas y, si se permite formato enriquecido, sanear con una librería robusta (DOMPurify) con allow-list de etiquetas.

```
# VULNERABLE (PHP): imprime el comentario sin escapar
echo "<div>" . $comentario . "</div>";        // <- se interpreta como HTML

# CORREGIDO: escapar la salida a HTML
echo "<div>" . htmlspecialchars($comentario, ENT_QUOTES, 'UTF-8') . "</div>";
```

5.2 Defensa en profundidad

- **Tokens anti-CSRF** en `/comentarios.php` y toda acción que cambie estado, validados en el servidor.
- **Content Security Policy (CSP)** restrictiva (sin `inline`; `script-src` acotado) para frenar la ejecución del payload.
- **Cookies con** `HttpOnly` (inaccesibles a JavaScript), `Secure` y `SameSite`.
- **Validación de entrada** (allow-list) de los comentarios como control adicional.
- **Revisión de todos los puntos** que almacenan y muestran contenido de usuario (XSS almacenado, reflejado y DOM).

5.3 Plan de acción

Acción	Responsable	Prioridad	Plazo	Estado
Escapar la salida de los comentarios (y revisar otros puntos de renderizado)	Equipo de desarrollo	Crítica	Inmediato	Pendiente
Implementar tokens anti-CSRF en acciones de cambio de estado	Equipo de desarrollo	Alta	7–15 días	Pendiente
Implementar CSP y cookies HttpOnly/Secure/SameSite	Desarrollo / Infraestructura	Alta	7–15 días	Pendiente
Revisión del código por XSS (almacenado, reflejado, DOM) + saneamiento	Desarrollo / SecOps	Alta	15–30 días	Pendiente
Test de regresión (payloads XSS = escapados) y re-test	QA / Seguridad	Alta	Con el fix	Pendiente

5.4 Verificación (re-test)

Volver a publicar un comentario con `<img src=x onerror=...>` y confirmar que se muestra como texto literal (escapado) y no se ejecuta, que `/comentarios.php` rechaza peticiones sin token anti-CSRF válido, y que la CSP bloquea scripts no autorizados. Confirmar que el comentar legítimo sigue funcionando.

> ✔ Criterio de cierre: el ticket se marca como *Cerrado* cuando el re-test confirma que el contenido de comentarios se renderiza escapado, que no se ejecutan acciones en sesiones ajenas y que las acciones de estado exigen token anti-CSRF, sin regresiones en la funcionalidad legítima.

6. Referencias e Historial

Referencias

- OWASP Top 10 (2021) — A03 Injection: https://owasp.org/Top10/A03_2021-Injection/
- CWE-79 — Cross-site Scripting: https://cwe.mitre.org/data/definitions/79.html
- CWE-352 — Cross-Site Request Forgery (CSRF): https://cwe.mitre.org/data/definitions/352.html
- CWE-116 — Improper Encoding or Escaping of Output: https://cwe.mitre.org/data/definitions/116.html
- OWASP — Cross Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
- OWASP — CSRF Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
- PortSwigger Web Security Academy — Stored XSS: https://portswigger.net/web-security/cross-site-scripting/stored
- CVSS v3.1 Calculator (FIRST): https://www.first.org/cvss/calculator/3.1

Historial de cambios

Versión	Fecha	Autor	Cambio
1.0	2026-11-09	J. M. F.	Creación del reporte: XSS almacenado con ejecución de acción autenticada (publicación forzada de comentario) en el blog.

🔒 Documento confidencial — uso interno / cliente autorizado únicamente. Las pruebas se realizaron en un entorno de laboratorio con autorización explícita.